

# CSDS 233 Programming Assignment 1: Interactive Music Sequencer

Name: `TODO` · Case ID: `TODO`

## 1. Introduction

### 1.1 The project

The sequencer stores a piece of music as a **doubly linked list of phrases**. Each node holds one phrase, a string of `PITCH:DURATION` tokens such as `C4:0.5 E4:0.5 G4:1.0` (chords are written `C4+E4+G4:1.0`, rests `R:0.5`). A console menu lets the user build and rearrange the composition (insert at the front, back, after the current phrase or at an index; repeat, remove and move phrases), move a "current" cursor through it, and play it with the provided `MidiPlayer`.

All of the list code is my own. No `java.util` class is used: no collections, no `Scanner` (input is read with `BufferedReader`), and no array used as a substitute for a list. The only `java.util` import in the project is in the instructor's unmodified `MidiPlayer.java`.

### 1.2 Design

| File                               | Responsibility                                                                                                                                                                                                                                                                                                                                                                           |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>DoublyLinkedList.java</code> | The required interface: <code>insertFirst</code> , <code>insertLast</code> , <code>insertAt</code> , <code>removeAt</code> , <code>get</code> , <code>size</code> , <code>isEmpty</code> .                                                                                                                                                                                               |
| <code>Node.java</code>             | One node with explicit data, <code>prev</code> and <code>next</code> fields.                                                                                                                                                                                                                                                                                                             |
| <code>PhraseList.java</code>       | <code>PhraseList&lt;E&gt;</code> implements <code>DoublyLinkedList&lt;E&gt;</code> . Keeps <code>head</code> , <code>tail</code> , <code>size</code> and <code>current</code> , and provides the cursor operations, <code>move</code> , traversal for playback, printing ( <code>toString</code> , <code>status</code> , <code>backwardTrace</code> ) and <code>checkInvariants</code> . |
| <code>PhraseValidator.java</code>  | Checks every phrase before it is stored, so <code>MidiPlayer</code> never sees malformed input.                                                                                                                                                                                                                                                                                          |
| <code>Main.java</code>             | The menu-driven driver.                                                                                                                                                                                                                                                                                                                                                                  |
| <code>TestRunner.java</code>       | Headless test harness (no JUnit, no sound).                                                                                                                                                                                                                                                                                                                                              |
| <code>MidiPlayer.java</code>       | Provided by the instructor, unchanged.                                                                                                                                                                                                                                                                                                                                                   |

Main design decisions:

- **All link changes go through four private helpers:** `linkIntoEmpty`, `linkBefore`, `linkAfter` and `detach`. Each one updates `head`, `tail` and `size` itself, so the special cases (empty list, new head, new tail) are handled in one place. Every insertion, removal and move is built from these helpers, which keeps `head`, `tail` and `size` correct after every operation.
- **Index errors throw `IndexOutOfBoundsException` before anything is modified.** The message names the index, the size and the valid range, e.g. `Index 3 is out of bounds for size 3 (valid range: 0..2)`. A rejected call therefore leaves the list exactly as it was; the tests check this, including that no node was allocated.
- **Lookups walk from the closer end.** `get`, `removeAt` and `insertAt` find their node with `nodeAt(i)`, which starts at the head when `i < size/2` and at the tail otherwise, so it takes at most about  $n/2$  hops.

- **Traversal without `java.util.Iterator`**. `PhraseList` declares its own callback interface `Visitor<E> { boolean visit(int index, E element); }`. The driver passes a lambda that prints `Playing node [i]: ...` and calls `player.play(...)`; returning `false` stops playback early, for example when no MIDI device is available.
- **null elements are rejected**, so a `null` return value always means "nothing", e.g. `removeCurrent()` on an empty list.

### 1.3 Why `current` lives in the list class

The cursor has a strong invariant: `current == null` exactly when the list is empty, and otherwise `current` is a node that is in the list. Only the class that rewires links can keep that true, because the cursor has to be fixed *in the same step* that a node is unlinked. `unlink(node)` therefore moves the cursor first (to `next`, or to `prev` if the tail is being removed, or to `null` if it was the last node) and only then calls `detach`.

If the driver kept the cursor instead, every operation could quietly break it. An index cursor shifts whenever something is inserted or removed in front of it, or moved past it. A node reference held by the driver would point at a detached node after a removal, and it would also expose the private `Node` class outside the list. Keeping `current` inside `PhraseList` also makes the cursor operations  $O(1)$ : `insertAfterCurrent`, `removeCurrent`, `repeatCurrent`, `moveForward` and `moveBackward` only touch the current node's neighbours.

Storing `current` as a **node reference** rather than an index is what makes `move` simple: the moved node is the same object before and after, so the cursor needs no update at all.

### 1.4 How `move(sourceIndex, targetIndex)` relinks nodes

`targetIndex` is the final position of the moved node. The implementation never allocates a node and never assigns data; it only rewrites `prev/next` links:

1. **Validate** both indices against `0..size-1`. If either is invalid, throw `IndexOutOfBoundsException`; nothing has been touched yet. If `sourceIndex == targetIndex`, return `false` (no-op).
2. **Find** the node: `node = nodeAt(sourceIndex)`.
3. **Detach** it: `pred.next = succ` (or `head = succ`), `succ.prev = pred` (or `tail = pred`), clear the node's own links, `size--`. The other  $n-1$  nodes now form a valid list on their own.
4. **Re-attach** it so that its index becomes `targetIndex` in the full list. If `targetIndex == size` (the shortened size), it becomes the new tail: `linkAfter(node, tail)`. Otherwise it goes directly in front of the node that is now at `targetIndex`: `linkBefore(node, nodeAt(targetIndex))`.

Example, the assignment's case `move(2, 0)` on `[C4:1.0, E4:1.0, G4:2.0]`:

|                                 |                                           |                                                                    |
|---------------------------------|-------------------------------------------|--------------------------------------------------------------------|
| <code>detach(G4)</code>         | <code>C4 &lt;-&gt; E4</code>              | <code>tail = E4, E4.next = null</code>                             |
| <code>linkBefore(G4, C4)</code> | <code>G4 &lt;-&gt; C4 &lt;-&gt; E4</code> | <code>G4.prev = null, G4.next = C4, C4.prev = G4, head = G4</code> |

Detaching first and then re-inserting means **adjacent nodes, head/tail moves and size-2 lists need no special code**. After the detach the neighbours are already linked to each other, and `linkBefore/ linkAfter` already handle a new head or tail. (Swapping links in place would need separate code for adjacent nodes, which is a common source of bugs.) A size-1 list has only one valid pair, `(0, 0)`, which is the no-op. `current` is never touched, and because the node object is reused it still points at the same phrase, now possibly at a different index.

## 1.5 Phrase validation

`MidiPlayer.play()` calls `Double.parseDouble` on every duration and does not catch the exception, so a phrase like `C4:abc` would throw `NumberFormatException` and crash the program halfway through playback.

`PhraseValidator.validate()` therefore checks each phrase **before** it is stored. It scans the text character by character (no `split`, no regular expressions) against this grammar:

```
token    = pitches ":" duration
pitches  = "R" | pitch { "+" pitch }      R = rest, "+" joins chord notes
pitch    = letter [ "#" | "b" ] octave    C4, D#4, Eb5 (MIDI note must be <= 127)
letter   = "A" .. "G"                   octave = "0" .. "9"
duration = positive decimal, e.g. 1, 0.5, .25 (at most 60 seconds)
```

An invalid phrase is rejected with a message that names the token, e.g. `token 2 'E4:abc' is invalid: duration 'abc' is not a positive decimal number`, and nothing is inserted. Valid phrases are stored with their whitespace normalized. The 60-second limit is my own addition: `MidiPlayer` sleeps for `duration × 1000` ms, so a typo such as `C4:100000` would otherwise freeze the program for more than a day.

## 2. Complexity analysis

$n$  = number of phrases in the list,  $i$  = index argument,  $L$  = length of a phrase in characters. "Best" and "Worst" differ only where noted.

| Method                                     | Best                         | Worst                        | Justification                                                                                                                        |
|--------------------------------------------|------------------------------|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <code>PhraseList()</code>                  | $O(1)$                       | $O(1)$                       | Sets four fields.                                                                                                                    |
| <code>insertFirst(e)</code>                | $O(1)$                       | $O(1)$                       | head is known; constant number of link updates.                                                                                      |
| <code>insertLast(e)</code>                 | $O(1)$                       | $O(1)$                       | tail is kept, so no walk is needed.                                                                                                  |
| <code>insertAt(i, e)</code>                | $O(1)$ at $i = 0$ or $i = n$ | $O(n)$ for $i$ in the middle | $i = n$ is <code>insertLast</code> ; otherwise <code>nodeAt(i)</code> walks $\min(i, n-1-i) \leq n/2$ hops, then linking is $O(1)$ . |
| <code>removeAt(i)</code>                   | $O(1)$ at either end         | $O(n)$ in the middle         | Same walk as <code>get</code> ; unlinking is $O(1)$ .                                                                                |
| <code>get(i)</code>                        | $O(1)$ at either end         | $O(n)$ in the middle         | Walks from the closer end: $\min(i, n-1-i)$ hops (test T-42 measures this).                                                          |
| <code>size(), isEmpty()</code>             | $O(1)$                       | $O(1)$                       | size field.                                                                                                                          |
| <code>getCurrent()</code>                  | $O(1)$                       | $O(1)$                       | Reads <code>current.data</code> .                                                                                                    |
| <code>getCurrentIndex()</code>             | $O(1)$ (current is head)     | $O(n)$                       | Counts nodes from the head until it reaches current.                                                                                 |
| <code>insertAfterCurrent(e)</code>         | $O(1)$                       | $O(1)$                       | <code>linkAfter(node, current)</code> .                                                                                              |
| <code>repeatCurrent()</code>               | $O(1)$                       | $O(1)$                       | New node with the same phrase, <code>linkAfter(node, tail)</code> .                                                                  |
| <code>removeCurrent()</code>               | $O(1)$                       | $O(1)$                       | Both neighbours are known; the cursor moves to one of them.                                                                          |
| <code>moveForward(), moveBackward()</code> | $O(1)$                       | $O(1)$                       | Follow one link.                                                                                                                     |

| Method                                                                                         | Best                                                                        | Worst       | Justification                                                                                                          |
|------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|-------------|------------------------------------------------------------------------------------------------------------------------|
| <code>move(s, t)</code>                                                                        | $O(1)$ when $s == t$ , an index is invalid, or both indices are at the ends | $O(n)$      | Up to two <code>nodeAt</code> walks of $\leq n/2$ hops each; detaching and re-linking are $O(1)$ . No allocation.      |
| <code>clear()</code>                                                                           | $O(1)$                                                                      | $O(1)$      | Drops the references to head, tail and current; the garbage collector frees the nodes.                                 |
| <code>traverseAll(v)</code>                                                                    | $O(k)$ if the visitor stops after $k$ nodes                                 | $O(n)$      | One pass from head to tail (plus the visitor's own cost).                                                              |
| <code>traverseFromCurrent(v)</code>                                                            | $O(1)$ on an empty list                                                     | $O(n)$      | <code>getCurrentIndex()</code> plus one pass from current to tail.                                                     |
| <code>toString()</code> , <code>backwardTrace()</code>                                         | $O(n)$                                                                      | $O(n)$      | One full forward or backward pass (output length is proportional to the phrases' text).                                |
| <code>status()</code>                                                                          | $O(1)$                                                                      | $O(1)$      | Reads size, head, tail and current only.                                                                               |
| <code>checkInvariants()</code>                                                                 | $O(1)$ on an empty list                                                     | $O(n)$      | One forward and one backward pass, each capped at size + 1 steps so it terminates even if the links contain a cycle.   |
| <code>nodeAt(i)</code> (package-private)                                                       | $O(1)$                                                                      | $O(n)$      | $\min(i, n-1-i)$ hops.                                                                                                 |
| <code>PhraseValidator.validate(p)</code> , <code>isValid(p)</code> , <code>normalize(p)</code> | $O(L)$                                                                      | $O(L)$      | Each character is looked at a constant number of times.                                                                |
| <code>MidiPlayer.play(p)</code> (provided)                                                     | $O(T)$ work for $T$ tokens                                                  | $O(T)$ work | CPU work is tiny; <b>wall-clock time equals the sum of the note durations</b> because the player sleeps for each note. |

Driver operations (menu options):

| Menu operation                             | Cost                                                | Notes                                                                                                                 |
|--------------------------------------------|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Insert / remove / move / repeat / navigate | cost of the list method above, + $O(n)$             | After every change the driver prints the list and status, which is one $O(n)$ pass.                                   |
| Play all                                   | $O(n)$ list work + $O(\text{total tokens})$         | Wall time is the sum of all durations; for the demo composition that is 10 s, against microseconds for the traversal. |
| Play from current                          | $O(n)$ list work + $O(\text{tokens played})$        | <code>getCurrentIndex()</code> for the printed index, then one pass to the tail.                                      |
| Play current                               | $O(n)$ list work + $O(\text{tokens in one phrase})$ | $O(n)$ only because the printed node index is computed by walking from the head.                                      |
| Show list                                  | $O(n)$                                              | List, status, backward trace and an indexed view, each one pass.                                                      |
| Load demo composition                      | $O(1)$                                              | <code>clear()</code> plus five <code>insertLast</code> calls (a fixed number).                                        |

## 3. Testing

### 3.1 Approach

`TestRunner.java` is a plain Java program (no JUnit) that never creates a `MidiPlayer`, so it runs anywhere, including without a sound card: `javac *.java && java TestRunner`. It prints every test in the required block format, writes the full output to `test_log.txt` and this matrix to `test_matrix.md`, and exits with status 1 if any test fails. The whole suite of 52 tests runs in well under a second.

Each test builds its initial list, prints it, runs the operation(s) under test, and checks the result at several levels:

1. **Contents and cursor.** The full list and the current phrase are compared with the expected values after each operation.
2. **checkInvariants() after every single operation**, including every `insertLast / moveForward` used to build the initial list and each of the 50 moves in T-16. It checks `head.prev == null`, `tail.next == null`, `a.next.prev == a` for every node, that the forward and backward node counts equal `size`, and that `current` is in the list.
3. **Node identity for move.** Before a move, the test stores the node objects themselves in a second list (a `PhraseList<Node<String>>`, so even the tests use no `java.util`). Afterwards it checks that each position holds exactly the expected original object, and that no node's `data` reference changed. This proves the nodes were relinked, not copied and not data-swapped.
4. **No allocation.** `Node` counts its instances (a package-private test hook), and every move test checks that the count did not change.
5. **Invalid indices.** For each invalid call the test expects an `IndexOutOfBoundsException` with a non-empty message, then checks that the list's text, status and node objects are unchanged and that no node was allocated. Every index method is tested with negative, `== size` and `> size` arguments (for `insertAt`, where `size` is valid, with `size + 1` and larger).
6. **Walk direction.** `nodeAt` records how many links it followed. T-42 checks that `get(i)` on 7 elements takes `min(i, 6-i)` hops.
7. **Playback logic without sound.** The play-all and play-from-current paths use the same `traverseAll / traverseFromCurrent` calls as the driver, with a recording visitor instead of the MIDI player.
8. **Validation.** 24 malformed phrases (including the `C4:abc` case that crashes `MidiPlayer`) must be rejected with a message that names the bad token, and 8 valid ones (sharps, flats, rests, chords, extra whitespace, range limits) must be accepted.

**Do the tests catch real bugs?** To check that, I injected 19 typical linked-list bugs into copies of the code, one at a time, and re-ran the suite. All 19 made at least one test fail:

| #    | Injected bug                               | File            | Result | Failing tests                              |
|------|--------------------------------------------|-----------------|--------|--------------------------------------------|
| M-01 | move swaps data instead of relinking       | PhraseList.java | Caught | 11: T-02, T-03, T-04, T-05, T-06, T-07 ... |
| M-02 | move allocates a replacement node          | PhraseList.java | Caught | 11: T-02, T-03, T-04, T-05, T-06, T-07 ... |
| M-03 | move forgets the new-tail case             | PhraseList.java | Caught | 3: T-03, T-11, T-16                        |
| M-04 | removing tail current sets current to null | PhraseList.java | Caught | 3: T-27, T-29, T-33                        |
| M-05 | removeCurrent always moves to prev         | PhraseList.java | Caught | 2: T-26, T-32                              |
| M-06 | nodeAt always walks from head              | PhraseList.java | Caught | 1: T-42                                    |
| M-07 | insertAt accepts size+1                    | PhraseList.java | Caught | 1: T-40                                    |
| M-08 | linkAfter forgets succ.prev                | PhraseList.java | Caught | 1: T-21                                    |
| M-09 | detach forgets tail update                 | PhraseList.java | Caught | 9: T-02, T-07, T-10, T-16, T-27, T-29 ...  |
| M-10 | moveForward ignores the tail boundary      | PhraseList.java | Caught | 2: T-43, T-44                              |
| M-11 | removeCurrent on empty dereferences null   | PhraseList.java | Caught | 1: T-31                                    |
| M-12 | repeatCurrent inserts after current        | PhraseList.java | Caught | 1: T-24                                    |

| #    | Injected bug                                                    | File                 | Result | Failing tests                              |
|------|-----------------------------------------------------------------|----------------------|--------|--------------------------------------------|
| M-13 | traverseFromCurrent starts at head                              | PhraseList.java      | Caught | 1: T-47                                    |
| M-14 | insert into empty list leaves current null                      | PhraseList.java      | Caught | 45: T-01, T-02, T-03, T-04, T-05, T-06 ... |
| M-15 | insertAfterCurrent moves current                                | PhraseList.java      | Caught | 2: T-21, T-22                              |
| M-16 | element-index check accepts index == size (get, removeAt, move) | PhraseList.java      | Caught | 7: T-13, T-14, T-15, T-36, T-38, T-39 ...  |
| M-17 | validator accepts any duration                                  | PhraseValidator.java | Caught | 1: T-50                                    |
| M-18 | validator skips chord notes after the first                     | PhraseValidator.java | Caught | 1: T-50                                    |
| M-19 | validator ignores MIDI range                                    | PhraseValidator.java | Caught | 1: T-50                                    |

The driver was tested separately by piping scripted input into `java Main --echo` (the flag echoes each input line so the transcript is readable). The script loads the demo, moves (valid, same index, out of range), removes, enters an invalid phrase, a non-numeric index, an empty line and an unknown menu choice, and finally closes the input stream (EOF). The program never crashed; an excerpt is in 3.4. On a machine without a MIDI device, `MidiPlayer` cannot open a synthesizer and `play()` throws a `NullPointerException`. The driver catches that, prints a warning and keeps running.

### 3.2 Test matrix (from the actual run)

Notation: `[a, b, c] cur=x` is the list from head to tail and the current phrase. The *Expected* column was written in `TestRunner` before the run; the *Actual* column is generated from the list's state after the run, plus short notes that the test records while its checks run.

| Test ID | Category | Initial State                                                    | Action                            | Expected                                                                       | Actual                                                                                                                                                                                                                          | Status |
|---------|----------|------------------------------------------------------------------|-----------------------------------|--------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| T-01    | Insert   | <code>[] cur=null</code>                                         | <code>insertLast("C4:1.0")</code> | <code>[C4:1.0] cur=C4:1.0; head == tail == current</code>                      | <code>[C4:1.0] cur=C4:1.0; head == tail == current (same node)</code>                                                                                                                                                           | PASS   |
| T-02    | Move     | <code>[C4:1.0, E4:1.0, G4:2.0] cur=E4:1.0</code>                 | <code>move(2, 0)</code>           | <code>[G4:2.0, C4:1.0, E4:1.0] cur=E4:1.0; same nodes, none allocated</code>   | <code>[G4:2.0, C4:1.0, E4:1.0] cur=E4:1.0; same 3 node objects now in original order 2 0 1; 0 nodes allocated; no data swapped; current is the same node</code>                                                                 | PASS   |
| T-03    | Move     | <code>[C4:1.0, E4:1.0, G4:2.0] cur=E4:1.0</code>                 | <code>move(0, 2)</code>           | <code>[E4:1.0, G4:2.0, C4:1.0] cur=E4:1.0; same nodes, none allocated</code>   | <code>[E4:1.0, G4:2.0, C4:1.0] cur=E4:1.0; same 3 node objects now in original order 1 2 0; 0 nodes allocated; no data swapped; current is the same node</code>                                                                 | PASS   |
| T-04    | Move     | <code>[C4:1.0, D4:1.0, E4:1.0, F4:1.0] cur=E4:1.0</code>         | <code>move(1, 2)</code>           | <code>[C4:1.0, E4:1.0, D4:1.0, F4:1.0] cur=E4:1.0</code>                       | <code>[C4:1.0, E4:1.0, D4:1.0, F4:1.0] cur=E4:1.0; same 4 node objects now in original order 0 2 1 3; 0 nodes allocated; no data swapped; current is the same node</code>                                                       | PASS   |
| T-05    | Move     | <code>[C4:1.0, D4:1.0, E4:1.0, F4:1.0] cur=D4:1.0</code>         | <code>move(2, 1)</code>           | <code>[C4:1.0, E4:1.0, D4:1.0, F4:1.0] cur=D4:1.0</code>                       | <code>[C4:1.0, E4:1.0, D4:1.0, F4:1.0] cur=D4:1.0; same 4 node objects now in original order 0 2 1 3; 0 nodes allocated; no data swapped; current is the same node</code>                                                       | PASS   |
| T-06    | Move     | <code>[C4:1.0, E4:1.0, G4:2.0] cur=C4:1.0</code>                 | <code>move(0, 1)</code>           | <code>[E4:1.0, C4:1.0, G4:2.0] cur=C4:1.0; head updated</code>                 | <code>[E4:1.0, C4:1.0, G4:2.0] cur=C4:1.0; same 3 node objects now in original order 1 0 2; 0 nodes allocated; no data swapped; current is the same node</code>                                                                 | PASS   |
| T-07    | Move     | <code>[C4:1.0, E4:1.0, G4:2.0] cur=G4:2.0</code>                 | <code>move(2, 1)</code>           | <code>[C4:1.0, G4:2.0, E4:1.0] cur=G4:2.0; tail updated</code>                 | <code>[C4:1.0, G4:2.0, E4:1.0] cur=G4:2.0; same 3 node objects now in original order 0 2 1; 0 nodes allocated; no data swapped; current is the same node</code>                                                                 | PASS   |
| T-08    | Move     | <code>[C4:1.0, D4:1.0, E4:1.0, F4:1.0, G4:1.0] cur=D4:1.0</code> | <code>move(1, 3)</code>           | <code>[C4:1.0, E4:1.0, F4:1.0, D4:1.0, G4:1.0] cur=D4:1.0 (now index 3)</code> | <code>[C4:1.0, E4:1.0, F4:1.0, D4:1.0, G4:1.0] cur=D4:1.0; same 5 node objects now in original order 0 2 3 1 4; 0 nodes allocated; no data swapped; current is the same node; current followed the moved node to index 3</code> | PASS   |
| T-09    | Move     | <code>[C4:1.0, D4:1.0, E4:1.0, F4:1.0, G4:1.0] cur=G4:1.0</code> | <code>move(3, 1)</code>           | <code>[C4:1.0, F4:1.0, D4:1.0, E4:1.0, G4:1.0] cur=G4:1.0</code>               | <code>[C4:1.0, F4:1.0, D4:1.0, E4:1.0, G4:1.0] cur=G4:1.0; same 5 node objects now in original order 0 3 1 2 4; 0 nodes allocated; no data swapped; current is the same node</code>                                             | PASS   |
| T-10    | Move     | <code>[C4:1.0, E4:1.0] cur=C4:1.0</code>                         | <code>move(1, 0)</code>           | <code>[E4:1.0, C4:1.0] cur=C4:1.0; head and tail swapped</code>                | <code>[E4:1.0, C4:1.0] cur=C4:1.0; same 2 node objects now in original order 1 0; 0 nodes allocated; no data swapped; current is the same node</code>                                                                           | PASS   |

| Test ID | Category | Initial State                                          | Action                                                                         | Expected                                                                                    | Actual                                                                                                                                                                                      | Status |
|---------|----------|--------------------------------------------------------|--------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| T-11    | Move     | [C4:1.0, E4:1.0]<br>cur=E4:1.0                         | move(0, 1)                                                                     | [E4:1.0, C4:1.0] cur=E4:1.0;<br>head and tail swapped                                       | [E4:1.0, C4:1.0] cur=E4:1.0; same 2 node objects<br>now in original order 1 0; 0 nodes allocated; no data<br>swapped; current is the same node                                              | PASS   |
| T-12    | Move     | [C4:1.0, E4:1.0, G4:2.0]<br>cur=E4:1.0                 | move(0, 0); move(1, 1);<br>move(2, 2)                                          | [C4:1.0, E4:1.0, G4:2.0]<br>cur=E4:1.0; each call returns<br>false                          | [C4:1.0, E4:1.0, G4:2.0] cur=E4:1.0; all 3 calls<br>returned false; node order 0 1 2 unchanged                                                                                              | PASS   |
| T-13    | Move     | [C4:1.0] cur=C4:1.0                                    | move(0, 0); move(0, 1);<br>move(1, 0)                                          | [C4:1.0] cur=C4:1.0; move(0,0)<br>is a no-op, the others throw<br>IndexOutOfBoundsException | [C4:1.0] cur=C4:1.0; move(0,0) returned false; 2 x<br>IndexOutOfBoundsException, list unchanged                                                                                             | PASS   |
| T-14    | Move     | [] cur=null                                            | move(0, 0); move(-1, 0)                                                        | [] cur=null;<br>IndexOutOfBoundsException,<br>list unchanged                                | [] cur=null; 2 x IndexOutOfBoundsException, list<br>still empty                                                                                                                             | PASS   |
| T-15    | Move     | [C4:1.0, E4:1.0, G4:2.0]<br>cur=E4:1.0                 | move(-1, 0); move(3, 0);<br>move(4, 1); move(0, -1);<br>move(0, 3); move(1, 9) | [C4:1.0, E4:1.0, G4:2.0]<br>cur=E4:1.0; 6 x<br>IndexOutOfBoundsException,<br>same nodes     | [C4:1.0, E4:1.0, G4:2.0] cur=E4:1.0; 6 x<br>IndexOutOfBoundsException (negative, == size, ><br>size for source and target); list and nodes unchanged                                        | PASS   |
| T-16    | Move     | [C4:1.0, D4:1.0, E4:1.0,<br>F4:1.0, G4:1.0] cur=E4:1.0 | move(i, j) then move(j, i)<br>for every i, j in 0..4 (25<br>round trips)       | [C4:1.0, D4:1.0, E4:1.0, F4:1.0,<br>G4:1.0] cur=E4:1.0; original<br>nodes, 0 allocated      | [C4:1.0, D4:1.0, E4:1.0, F4:1.0, G4:1.0] cur=E4:1.0;<br>50 moves with invariants checked after each; moved<br>node always landed at the target; 0 nodes allocated;<br>final order 0 1 2 3 4 | PASS   |
| T-17    | Insert   | [] cur=null                                            | insertFirst("E4:1.0");<br>insertFirst("C4:1.0")                                | [C4:1.0, E4:1.0] cur=E4:1.0                                                                 | [C4:1.0, E4:1.0] cur=E4:1.0; first insert became<br>current; second insert left current alone                                                                                               | PASS   |
| T-18    | Insert   | [] cur=null                                            | insertAt(0, "C4:1.0")                                                          | [C4:1.0] cur=C4:1.0                                                                         | [C4:1.0] cur=C4:1.0; new node is head, tail and<br>current                                                                                                                                  | PASS   |
| T-19    | Insert   | [D4:1.0, F4:1.0]<br>cur=D4:1.0                         | insertAt(0, "C4:1.0");<br>insertAt(2, "E4:1.0");<br>insertAt(4, "G4:1.0")      | [C4:1.0, D4:1.0, E4:1.0, F4:1.0,<br>G4:1.0] cur=D4:1.0                                      | [C4:1.0, D4:1.0, E4:1.0, F4:1.0, G4:1.0] cur=D4:1.0;<br>head, middle and tail inserts correct; current stayed<br>on D4:1.0                                                                  | PASS   |
| T-20    | Cursor   | [] cur=null                                            | insertAfterCurrent("C4:1.0<br>")                                               | [C4:1.0] cur=C4:1.0 (only node<br>and current)                                              | [C4:1.0] cur=C4:1.0; new node is the only node and<br>current                                                                                                                               | PASS   |
| T-21    | Cursor   | [C4:1.0, G4:2.0]<br>cur=C4:1.0                         | insertAfterCurrent("E4:1.0<br>")                                               | [C4:1.0, E4:1.0, G4:2.0]<br>cur=C4:1.0                                                      | [C4:1.0, E4:1.0, G4:2.0] cur=C4:1.0; inserted at<br>index 1; current unchanged                                                                                                              | PASS   |
| T-22    | Cursor   | [C4:1.0, E4:1.0]<br>cur=E4:1.0                         | insertAfterCurrent("G4:2.0<br>")                                               | [C4:1.0, E4:1.0, G4:2.0]<br>cur=E4:1.0; Tail: G4:2.0                                        | [C4:1.0, E4:1.0, G4:2.0] cur=E4:1.0; tail is the new<br>node                                                                                                                                | PASS   |
| T-23    | Cursor   | [C4:1.0] cur=C4:1.0                                    | repeatCurrent()                                                                | [C4:1.0, C4:1.0] cur=C4:1.0<br>(original head); copy is a new<br>node                       | [C4:1.0, C4:1.0] cur=C4:1.0; returned true; two<br>distinct nodes; current still index 0                                                                                                    | PASS   |
| T-24    | Cursor   | [C4:1.0, E4:1.0, G4:2.0]<br>cur=E4:1.0                 | repeatCurrent()                                                                | [C4:1.0, E4:1.0, G4:2.0, E4:1.0]<br>cur=E4:1.0 (index 1)                                    | [C4:1.0, E4:1.0, G4:2.0, E4:1.0] cur=E4:1.0; copy<br>appended as new tail; current still index 1                                                                                            | PASS   |
| T-25    | Cursor   | [] cur=null                                            | repeatCurrent()                                                                | [] cur=null; returns false                                                                  | [] cur=null; returned false, no exception                                                                                                                                                   | PASS   |
| T-26    | Remove   | [C4:1.0, E4:1.0, G4:2.0]<br>cur=E4:1.0                 | removeCurrent()                                                                | [C4:1.0, G4:2.0] cur=G4:2.0;<br>returns E4:1.0                                              | [C4:1.0, G4:2.0] cur=G4:2.0; returned E4:1.0                                                                                                                                                | PASS   |
| T-27    | Remove   | [C4:1.0, E4:1.0, G4:2.0]<br>cur=G4:2.0                 | removeCurrent()                                                                | [C4:1.0, E4:1.0] cur=E4:1.0;<br>returns G4:2.0                                              | [C4:1.0, E4:1.0] cur=E4:1.0; returned G4:2.0                                                                                                                                                | PASS   |
| T-28    | Remove   | [C4:1.0, E4:1.0, G4:2.0]<br>cur=C4:1.0                 | removeCurrent()                                                                | [E4:1.0, G4:2.0] cur=E4:1.0;<br>returns C4:1.0                                              | [E4:1.0, G4:2.0] cur=E4:1.0; returned C4:1.0                                                                                                                                                | PASS   |
| T-29    | Remove   | [C4:1.0, E4:1.0]<br>cur=E4:1.0                         | removeCurrent()                                                                | [C4:1.0] cur=C4:1.0; head ==<br>tail                                                        | [C4:1.0] cur=C4:1.0; returned E4:1.0                                                                                                                                                        | PASS   |
| T-30    | Remove   | [C4:1.0] cur=C4:1.0                                    | removeCurrent()                                                                | [] cur=null; returns C4:1.0; head<br>= tail = null                                          | [] cur=null; returned C4:1.0                                                                                                                                                                | PASS   |
| T-31    | Remove   | [] cur=null                                            | removeCurrent()                                                                | [] cur=null; returns null, no<br>exception                                                  | [] cur=null; returned null, no exception                                                                                                                                                    | PASS   |
| T-32    | Remove   | [C4:1.0, E4:1.0, G4:2.0]<br>cur=E4:1.0                 | removeAt(1)                                                                    | [C4:1.0, G4:2.0] cur=G4:2.0                                                                 | [C4:1.0, G4:2.0] cur=G4:2.0; removeAt(1) returned<br>E4:1.0                                                                                                                                 | PASS   |
| T-33    | Remove   | [C4:1.0, E4:1.0, G4:2.0]<br>cur=G4:2.0                 | removeAt(2)                                                                    | [C4:1.0, E4:1.0] cur=E4:1.0                                                                 | [C4:1.0, E4:1.0] cur=E4:1.0; removeAt(2) returned<br>G4:2.0                                                                                                                                 | PASS   |
| T-34    | Remove   | [C4:1.0, E4:1.0, G4:2.0]<br>cur=E4:1.0                 | removeAt(0); removeAt(1)                                                       | [E4:1.0] cur=E4:1.0                                                                         | [E4:1.0] cur=E4:1.0; removeAt(0) returned C4:1.0;<br>removeAt(1) returned G4:2.0                                                                                                            | PASS   |
| T-35    | Remove   | [C4:1.0, E4:1.0]<br>cur=C4:1.0                         | removeAt(0)                                                                    | [E4:1.0] cur=E4:1.0; head ==<br>tail                                                        | [E4:1.0] cur=E4:1.0; removeAt(0) returned C4:1.0;<br>remaining node has prev == next == null                                                                                                | PASS   |
| T-36    | Remove   | [] cur=null                                            | removeAt(0)                                                                    | [] cur=null;<br>IndexOutOfBoundsException                                                   | [] cur=null; IndexOutOfBoundsException, list still<br>empty                                                                                                                                 | PASS   |

| Test ID | Category   | Initial State                                                             | Action                                                                                       | Expected                                                                                         | Actual                                                                                                                                                                                                                                          | Status |
|---------|------------|---------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| T-37    | Remove     | [C4:1.0, E4:1.0, G4:2.0]<br>cur=E4:1.0                                    | clear()                                                                                      | [] cur=null; Size: 0                                                                             | [] cur=null; all fields reset                                                                                                                                                                                                                   | PASS   |
| T-38    | Bounds     | [C4:1.0, E4:1.0, G4:2.0]<br>cur=E4:1.0                                    | get(-1); get(3); get(4)                                                                      | [C4:1.0, E4:1.0, G4:2.0]<br>cur=E4:1.0; 3 x<br>IndexOutOfBoundsException                         | [C4:1.0, E4:1.0, G4:2.0] cur=E4:1.0; 3 x<br>IndexOutOfBoundsException, list unchanged                                                                                                                                                           | PASS   |
| T-39    | Bounds     | [C4:1.0, E4:1.0, G4:2.0]<br>cur=E4:1.0                                    | removeAt(-1);<br>removeAt(3); removeAt(4)                                                    | [C4:1.0, E4:1.0, G4:2.0]<br>cur=E4:1.0; 3 x<br>IndexOutOfBoundsException                         | [C4:1.0, E4:1.0, G4:2.0] cur=E4:1.0; 3 x<br>IndexOutOfBoundsException, list unchanged                                                                                                                                                           | PASS   |
| T-40    | Bounds     | [C4:1.0, E4:1.0, G4:2.0]<br>cur=E4:1.0                                    | insertAt(-1, "A4:1.0");<br>insertAt(4, "A4:1.0");<br>insertAt(10, "A4:1.0")                  | [C4:1.0, E4:1.0, G4:2.0]<br>cur=E4:1.0; 3 x<br>IndexOutOfBoundsException, 0<br>nodes allocated   | [C4:1.0, E4:1.0, G4:2.0] cur=E4:1.0; 3 x<br>IndexOutOfBoundsException, list unchanged, 0<br>nodes allocated (index == size is valid, see the<br>INSERTAT FRONT, MIDDLE AND END test)                                                            | PASS   |
| T-41    | Bounds     | [] cur=null                                                               | get(0); get(-1); get(1)                                                                      | [] cur=null; 3 x<br>IndexOutOfBoundsException                                                    | [] cur=null; 3 x IndexOutOfBoundsException, no<br>NullPointerException                                                                                                                                                                          | PASS   |
| T-42    | Access     | [C4:1.0, D4:1.0, E4:1.0,<br>F4:1.0, G4:1.0, A4:1.0,<br>B4:1.0] cur=C4:1.0 | get(i) for i = 0..6                                                                          | C4..B4 returned in order; link<br>hops 0 1 2 3 2 1 0                                             | [C4:1.0, D4:1.0, E4:1.0, F4:1.0, G4:1.0, A4:1.0,<br>B4:1.0] cur=C4:1.0; all 7 values correct; link hops 0<br>1 2 3 2 1 0 (never more than 3 for size 7)                                                                                         | PASS   |
| T-43    | Navigation | [C4:1.0, E4:1.0, G4:2.0]<br>cur=C4:1.0                                    | moveBackward();<br>moveForward();<br>moveForward();<br>moveBackward()                        | returns false, true, true, false,<br>true; cur=E4:1.0                                            | [C4:1.0, E4:1.0, G4:2.0] cur=E4:1.0; returned false,<br>true, true, false, true; stayed put at both ends                                                                                                                                        | PASS   |
| T-44    | Navigation | [C4:1.0] cur=C4:1.0                                                       | moveForward();<br>moveBackward()                                                             | both return false; cur=C4:1.0                                                                    | [C4:1.0] cur=C4:1.0; returned false, false                                                                                                                                                                                                      | PASS   |
| T-45    | Navigation | [] cur=null                                                               | moveForward();<br>moveBackward();<br>getCurrent();<br>getCurrentIndex()                      | false, false, null, -1; no<br>exception                                                          | [] cur=null; returned false, false, getCurrent() = null,<br>getCurrentIndex() = -1                                                                                                                                                              | PASS   |
| T-46    | Playback   | [C4:1.0, E4:1.0, G4:2.0]<br>cur=E4:1.0                                    | traverseAll(recorder)                                                                        | 3 visited: [0] C4:1.0 -> [1]<br>E4:1.0 -> [2] G4:2.0; cur<br>unchanged                           | [C4:1.0, E4:1.0, G4:2.0] cur=E4:1.0; 3 visited: [0]<br>C4:1.0 -> [1] E4:1.0 -> [2] G4:2.0                                                                                                                                                       | PASS   |
| T-47    | Playback   | [C4:1.0, E4:1.0, G4:2.0]<br>cur=E4:1.0                                    | traverseFromCurrent(recor<br>der)                                                            | 2 visited: [1] E4:1.0 -> [2]<br>G4:2.0                                                           | [C4:1.0, E4:1.0, G4:2.0] cur=E4:1.0; 2 visited: [1]<br>E4:1.0 -> [2] G4:2.0                                                                                                                                                                     | PASS   |
| T-48    | Playback   | [C4:1.0, E4:1.0, G4:2.0]<br>cur=C4:1.0                                    | traverseAll(recorder that<br>returns false at index 1)                                       | 2 visited: [0] C4:1.0 -> [1]<br>E4:1.0                                                           | [C4:1.0, E4:1.0, G4:2.0] cur=C4:1.0; 2 visited: [0]<br>C4:1.0 -> [1] E4:1.0 (the driver stops like this if<br>MIDI playback fails)                                                                                                              | PASS   |
| T-49    | Playback   | [] cur=null                                                               | traverseAll(recorder);<br>traverseFromCurrent(recor<br>der); getCurrent()                    | 0 visited by both; current null,<br>so play current has nothing to<br>play                       | [] cur=null; play all: 0 visited: (none); play from<br>current: 0 visited: (none); getCurrent() = null                                                                                                                                          | PASS   |
| T-50    | Validation | [C4:1.0] cur=C4:1.0                                                       | for 24 malformed phrases:<br>insertLast(p) only if<br>PhraseValidator.validate(p)<br>== null | [C4:1.0] cur=C4:1.0; 24/24<br>rejected, each message names<br>the bad token                      | [C4:1.0] cur=C4:1.0; 24/24 rejected, each message<br>names the bad token (listed before the summary);<br>list unchanged                                                                                                                         | PASS   |
| T-51    | Validation | [] cur=null                                                               | for 8 valid phrases:<br>insertLast(normalize(p))<br>after validate(p) == null                | [C4:1.0, D#4:0.5, Eb5:0.25,<br>R:0.5, C4+E4+G4:1.0, A4:0.5<br>Bb3:5, G9:1, Cb0:60]<br>cur=C4:1.0 | [C4:1.0, D#4:0.5, Eb5:0.25, R:0.5, C4+E4+G4:1.0,<br>A4:0.5 Bb3:5, G9:1, Cb0:60] cur=C4:1.0; 8/8<br>accepted; extra whitespace normalized                                                                                                        | PASS   |
| T-52    | Validation | [C4:1.0] cur=C4:1.0                                                       | Main.loadDemoCompositi<br>on(list)                                                           | 5 valid phrases; old content<br>replaced; cur = first phrase<br>(head)                           | [C4:0.5 C4:0.5 G4:0.5 G4:0.5, A4:0.5 A4:0.5<br>G4:1.0, F4:0.5 F4:0.5 E4:0.5 E4:0.5, D4:0.5 D4:0.5<br>C4:1.0, F4+A4+C5:0.5 G4+B4+D5:0.5<br>C4+E4+G4:1.0] cur=C4:0.5 C4:0.5 G4:0.5 G4:0.5;<br>5 phrases, all pass PhraseValidator; current = head | PASS   |

**Result: 52 of 52 tests passed, 0 failed (Java 21.0.11).**

### 3.3 Excerpts from `test_log.txt`

The assignment's example, an in-place move from tail to head:

```
=== RUNNING TEST T-02: IN-PLACE MOVE (TAIL TO HEAD) ===
Initial List: [ C4:1.0 ] <-> [ E4:1.0 ] <-> [ G4:2.0 ]
Size: 3 | Head: C4:1.0 | Tail: G4:2.0 | Current: E4:1.0
Executing: move(2, 0)
Result List: [ G4:2.0 ] <-> [ C4:1.0 ] <-> [ E4:1.0 ]
Size: 3 | Head: G4:2.0 | Tail: E4:1.0 | Current: E4:1.0
Check Backwards Link (Tail to Head): E4:1.0 -> C4:1.0 -> G4:2.0 -> null
[SUCCESS]
```

Invalid indices are rejected and the list is left unchanged:

```
=== RUNNING TEST T-15: MOVE WITH INVALID INDICES IS REJECTED ===
Initial List: [ C4:1.0 ] <-> [ E4:1.0 ] <-> [ G4:2.0 ]
Size: 3 | Head: C4:1.0 | Tail: G4:2.0 | Current: E4:1.0
Executing: move(-1, 0); move(3, 0); move(4, 1); move(0, -1); move(0, 3); move(1, 9)
Result List: [ C4:1.0 ] <-> [ E4:1.0 ] <-> [ G4:2.0 ]
Size: 3 | Head: C4:1.0 | Tail: G4:2.0 | Current: E4:1.0
Check Backwards Link (Tail to Head): G4:2.0 -> E4:1.0 -> C4:1.0 -> null
[SUCCESS]
```

Removing the current node at the tail moves the cursor back; removing the last node empties the list:

```
=== RUNNING TEST T-27: REMOVE CURRENT AT TAIL (CURRENT -> PREV) ===
Initial List: [ C4:1.0 ] <-> [ E4:1.0 ] <-> [ G4:2.0 ]
Size: 3 | Head: C4:1.0 | Tail: G4:2.0 | Current: G4:2.0
Executing: removeCurrent()
Result List: [ C4:1.0 ] <-> [ E4:1.0 ]
Size: 2 | Head: C4:1.0 | Tail: E4:1.0 | Current: E4:1.0
Check Backwards Link (Tail to Head): E4:1.0 -> C4:1.0 -> null
[SUCCESS]

=== RUNNING TEST T-30: REMOVE CURRENT ON SIZE 1 (LIST BECOMES EMPTY) ===
Initial List: [ C4:1.0 ]
Size: 1 | Head: C4:1.0 | Tail: C4:1.0 | Current: C4:1.0
Executing: removeCurrent()
Result List: (empty)
Size: 0 | Head: null | Tail: null | Current: null
Check Backwards Link (Tail to Head): null
[SUCCESS]
```

`get` walks from the closer end:

```
=== RUNNING TEST T-42: GET WALKS FROM THE CLOSER END ===
Initial List: [ C4:1.0 ] <-> [ D4:1.0 ] <-> [ E4:1.0 ] <-> [ F4:1.0 ] <-> [ G4:1.0 ] <-> [ A4:1.0 ] <-> [ B4:1.0 ]
Size: 7 | Head: C4:1.0 | Tail: B4:1.0 | Current: C4:1.0
Executing: get(i) for i = 0..6
Result List: [ C4:1.0 ] <-> [ D4:1.0 ] <-> [ E4:1.0 ] <-> [ F4:1.0 ] <-> [ G4:1.0 ] <-> [ A4:1.0 ] <-> [ B4:1.0 ]
Size: 7 | Head: C4:1.0 | Tail: B4:1.0 | Current: C4:1.0
Check Backwards Link (Tail to Head): B4:1.0 -> A4:1.0 -> G4:1.0 -> F4:1.0 -> E4:1.0 -> D4:1.0 -> C4:1.0 -> null
[SUCCESS]
```

Exception and validator messages recorded during the run (selection):

```
T-14 move(0, 0) -> IndexOutOfBoundsException: Source index 0 is out of bounds: the list is empty
T-15 move(3, 0) -> IndexOutOfBoundsException: Source index 3 is out of bounds for size 3 (valid range: 0..2)
T-15 move(0, -1) -> IndexOutOfBoundsException: Target index -1 is out of bounds for size 3 (valid range: 0..2)
T-36 removeAt(0) -> IndexOutOfBoundsException: Index 0 is out of bounds: the list is empty
T-38 get(3) -> IndexOutOfBoundsException: Index 3 is out of bounds for size 3 (valid range: 0..2)
T-40 insertAt(4, A4:1.0) -> IndexOutOfBoundsException: Insert index 4 is out of bounds for size 3 (valid range: 0..3)
T-50 "C4:abc" -> token 1 'C4:abc' is invalid: duration 'abc' is not a positive decimal number (e.g. 0.5 or 1.0)
T-50 "C4:1.0 E4:abc G4:1.0" -> token 2 'E4:abc' is invalid: duration 'abc' is not a positive decimal number (e.g. 0.5 or 1.0)
T-50 "H4:1.0" -> token 1 'H4:1.0' is invalid: pitch 'H4' must start with a note letter A-G (or be R for a rest)
T-50 "C4+R:1.0" -> token 1 'C4+R:1.0' is invalid: a rest (R) cannot be part of a chord
T-50 "G#9:1.0" -> token 1 'G#9:1.0' is invalid: pitch 'G#9' is above the MIDI range (highest note is G9)
T-50 "C4" -> token 1 'C4' is invalid: missing ':' between pitch and duration (expected PITCH:DURATION, e.g. C4:1.0)
T-50 "C4:1e3" -> token 1 'C4:1e3' is invalid: duration '1e3' is not a positive decimal number (e.g. 0.5 or 1.0)
T-50 "C4:100" -> token 1 'C4:100' is invalid: duration '100' is too long (maximum is 60.0)
T-50 "" -> the phrase is empty (expected tokens like C4:1.0 E4:0.5)
```

Summary at the end of the log (the middle of the per-test list is shortened here; the full list is in the matrix above):

```
=== SUMMARY ===
T-01 [SUCCESS] INSERT INTO EMPTY LIST SETS CURRENT
T-02 [SUCCESS] IN-PLACE MOVE (TAIL TO HEAD)
T-03 [SUCCESS] IN-PLACE MOVE (HEAD TO TAIL)
T-04 [SUCCESS] IN-PLACE MOVE (ADJACENT, FORWARD)
[... 45 more lines (T-05 to T-49), all [SUCCESS] ...]
T-50 [SUCCESS] INVALID PHRASES ARE REJECTED AND NOT INSERTED
T-51 [SUCCESS] VALID PHRASES (SHARPS, FLATS, RESTS, CHORDS) ARE ACCEPTED
T-52 [SUCCESS] DEMO COMPOSITION LOADS AND EVERY PHRASE IS VALID
Total: 52 | Passed: 52 | Failed: 0
ALL TESTS PASSED
```

### 3.4 Driver smoke test (excerpt)

Scripted input piped into `java Main --echo`; the menu is omitted after the first prompt.

```
Choose an option [0-15]: 15
Loaded the demo composition: 5 phrases.
List: [ C4:0.5 C4:0.5 G4:0.5 G4:0.5 ] <-> [ A4:0.5 A4:0.5 G4:1.0 ] <-> [ F4:0.5 F4:0.5 E4:0.5 E4:0.5 ] <-> [ D4:0.5 D4:0.5 C4:1.0 ] <-> [ F4+A4+C5:0.5 G4+B4+D5:0.5 C4+E4+G4:1.0 ]
Size: 5 | Head: C4:0.5 C4:0.5 G4:0.5 G4:0.5 | Tail: F4+A4+C5:0.5 G4+B4+D5:0.5 C4+E4+G4:1.0 | Current: C4:0.5 C4:0.5 G4:0.5 G4:0.5
Choose an option [0-15]: 8
Source index (0..4): 4
Target index, i.e. final position (0..4): 0
Moved the node at index 4 to index 0 (links rewired, nothing copied).
List: [ F4+A4+C5:0.5 G4+B4+D5:0.5 C4+E4+G4:1.0 ] <-> [ C4:0.5 C4:0.5 G4:0.5 G4:0.5 ] <-> [ A4:0.5 A4:0.5 G4:1.0 ] <-> [ F4:0.5 F4:0.5 E4:0.5 E4:0.5 ] <-> [ D4:0.5 D4:0.5 C4:1.0 ]
Size: 5 | Head: F4+A4+C5:0.5 G4+B4+D5:0.5 C4+E4+G4:1.0 | Tail: D4:0.5 D4:0.5 C4:1.0 | Current: C4:0.5 C4:0.5 G4:0.5 G4:0.5
Choose an option [0-15]: 8
```

```

Source index (0..4): 1
Target index, i.e. final position (0..4): 1
Source and target are the same: nothing to move.
List: [ F4+A4+C5:0.5 G4+B4+D5:0.5 C4+E4+G4:1.0 ] <-> [ C4:0.5 C4:0.5 G4:0.5 G4:0.5 ] <-> [ A4:0.5 A4:0.5 G4:1.0 ] <-> [ F4:0.5 F4:0.5 E4:0.5 E4:0.5 ] <-> [ D4:0.5 D4:0.5 C4:1.0 ]
Size: 5 | Head: F4+A4+C5:0.5 G4+B4+D5:0.5 C4+E4+G4:1.0 | Tail: D4:0.5 D4:0.5 C4:1.0 | Current: C4:0.5 C4:0.5 G4:0.5 G4:0.5
Choose an option [0-15]: 8
Source index (0..4): 0
Target index, i.e. final position (0..4): 9
[!] Warning: Target index 9 is out of bounds for size 5 (valid range: 0..4). The list was not changed.
Choose an option [0-15]: 6
Removed the current phrase: C4:0.5 C4:0.5 G4:0.5 G4:0.5
List: [ F4+A4+C5:0.5 G4+B4+D5:0.5 C4+E4+G4:1.0 ] <-> [ A4:0.5 A4:0.5 G4:1.0 ] <-> [ F4:0.5 F4:0.5 E4:0.5 E4:0.5 ] <-> [ D4:0.5 D4:0.5 C4:1.0 ]
Size: 4 | Head: F4+A4+C5:0.5 G4+B4+D5:0.5 C4+E4+G4:1.0 | Tail: D4:0.5 D4:0.5 C4:1.0 | Current: A4:0.5 A4:0.5 G4:1.0
Choose an option [0-15]: 7
Index to remove (0..3): 2
Removed the phrase at index 2: F4:0.5 F4:0.5 E4:0.5 E4:0.5
List: [ F4+A4+C5:0.5 G4+B4+D5:0.5 C4+E4+G4:1.0 ] <-> [ A4:0.5 A4:0.5 G4:1.0 ] <-> [ D4:0.5 D4:0.5 C4:1.0 ]
Size: 3 | Head: F4+A4+C5:0.5 G4+B4+D5:0.5 C4+E4+G4:1.0 | Tail: D4:0.5 D4:0.5 C4:1.0 | Current: A4:0.5 A4:0.5 G4:1.0
Choose an option [0-15]: 1
Enter phrase (e.g. C4:0.5 E4:0.5 G4:1.0): C4:abc
[!] Warning: Phrase rejected, token 1 'C4:abc' is invalid: duration 'abc' is not a positive decimal number (e.g. 0.5 or 1.0). Nothing was inserted.
Choose an option [0-15]: 2
Enter phrase (e.g. C4:0.5 E4:0.5 G4:1.0): E4:0.5 G4:0.5
Inserted at the back: E4:0.5 G4:0.5
List: [ F4+A4+C5:0.5 G4+B4+D5:0.5 C4+E4+G4:1.0 ] <-> [ A4:0.5 A4:0.5 G4:1.0 ] <-> [ D4:0.5 D4:0.5 C4:1.0 ] <-> [ E4:0.5 G4:0.5 ]
Size: 4 | Head: F4+A4+C5:0.5 G4+B4+D5:0.5 C4+E4+G4:1.0 | Tail: E4:0.5 G4:0.5 | Current: A4:0.5 A4:0.5 G4:1.0
Choose an option [0-15]: 7
Index to remove (0..3): abc
[!] Warning: 'abc' is not a valid whole number. Operation cancelled.
Choose an option [0-15]: 13
Current moved backward.
List: [ F4+A4+C5:0.5 G4+B4+D5:0.5 C4+E4+G4:1.0 ] <-> [ A4:0.5 A4:0.5 G4:1.0 ] <-> [ D4:0.5 D4:0.5 C4:1.0 ] <-> [ E4:0.5 G4:0.5 ]
Size: 4 | Head: F4+A4+C5:0.5 G4+B4+D5:0.5 C4+E4+G4:1.0 | Tail: E4:0.5 G4:0.5 | Current: F4+A4+C5:0.5 G4+B4+D5:0.5 C4+E4+G4:1.0
Choose an option [0-15]: 13
[!] Warning: Already at the first phrase (head): cannot move backward.
Size: 4 | Head: F4+A4+C5:0.5 G4+B4+D5:0.5 C4+E4+G4:1.0 | Tail: E4:0.5 G4:0.5 | Current: F4+A4+C5:0.5 G4+B4+D5:0.5 C4+E4+G4:1.0
Choose an option [0-15]:
[!] Warning: Please type a menu number (0-15).
Choose an option [0-15]: xyz
[!] Warning: 'xyz' is not a menu number. Please type 0-15.
Choose an option [0-15]:
Input closed (end of file). Exiting the sequencer...
MIDI system closed. Goodbye!

```

## 4. Reflection

### 4.1 Most interesting

TODO: write yourself

## 4.2 Most challenging

TODO: write yourself

## 4.3 Feedback

TODO: write yourself

## 5. Possible extensions

---

1. **Tempo (BPM).** Store durations in beats instead of seconds and add a global tempo; a beat lasts  $60 / \text{BPM}$  seconds. Because `MidiPlayer` must not change, the driver would rewrite each token's duration ( $\text{beats} \times 60 / \text{BPM}$ ) before calling `play()`. Changing the tempo would then be  $O(1)$  and would not touch the stored phrases.
2. **Transpose.** Shift the current phrase, or the whole list, by  $k$  semitones. The validator already computes each note's MIDI number, so transposing means parsing each pitch, adding  $k$ , checking  $0..127$  and spelling the note again (e.g. with sharps). Cost  $O(\text{total tokens})$ , done in one traversal.
3. **Undo / redo.** Keep two stacks of edit commands, built on the same doubly linked list. Every edit has a cheap inverse: `insertAt(i, p) ↔ removeAt(i), removeCurrent() ↔ insertAt(oldIndex, phrase)` plus restoring the cursor, and `move(s, t) ↔ move(t, s)` (test T-16 already shows that this round trip restores the exact same nodes).
4. **Save / load.** Write one phrase per line to a text file with `BufferedWriter`, and read it back with `BufferedReader`, running every line through `PhraseValidator` and reporting bad lines by line number instead of loading them. This is  $O(n + \text{total characters})$  and would let users keep their compositions between runs.
5. **Instrument change.** The player always uses program 0 (grand piano). An extension could store an instrument number per phrase (e.g. a prefix `@40` for violin) and call `MidiChannel.programChange(n)` before playing that node. Since `MidiPlayer` is fixed for this assignment, this would need a new player class.